PES UNIVERSITY

CELEBRATING 50 YEARS

Data Structures

Dilip Kumar Maripuri

Computer Applications

PES UNIVERSITY

Data Structures

Session : 2-3 Trees

Dilip Kumar Maripuri
Computer Applications

Data Structures

2-3 Trees

- A 2-3 tree is a type of balanced search tree, specifically a B-tree of order 3.
- It is structured in such a way that every non-root node has either two or three children.
- This self-balancing property makes it an efficient data structure for searching, insertion, and deletion operations.

Data Structures
Characteristics of 2-3 Trees

PES UNIVERSITY

- **Node Structure:**
  - Each node can contain **one or two keys**.
  - A node with one key has **two children**.
  - A node with two keys has **three children**.
- **4-Nodes (Temporary)**
  - During the insertion or deletion operations, a temporary 4-node can be created with three data elements and four children.
  - However, the 4-node doesn't persist; it is immediately split into two 2-nodes to preserve the tree's properties.

Data Structures
Characteristics of 2-3 Trees

PES UNIVERSITY

- **Balance Property:**
  - The tree remains **perfectly balanced**, meaning all leaf nodes exist at the same level.
  - The height of the tree is kept minimal, ensuring efficient searching.
- **Ordering Invariant:**
  - If a node contains **two keys**, say K_1 and K_2, it has three children:
    - The left child contains values **less than** K_1.
    - The middle child contains values **between** K_1 and K_2.
    - The right child contains values **greater than** K_2.

Data Structures

Why Use a 2-3 Tree?

- **Faster search times** compared to unbalanced binary search trees.
- **Automatic balancing** ensures that operations remain $O(\log n)$.
- Forms the foundation of more advanced structures like **2-3-4 trees** and **Red-Black Trees**.

Data Structures

Comparison with Other Trees

PES UNIVERSITY

Tree Type	Balance Property	Height Complexity	Use Case
BST (Binary Search Tree)	No balancing	$O(n)$ worst case	General usage, but can become inefficient if unbalanced
AVL Tree	Strict balancing	$O(\log n)$	Good for search-heavy applications
2-3 Tree	Perfect balancing	$O(\log n)$	More stable insertions/deletions, used in databases

- **Search for the Correct Position:**
  - Start at the root and traverse down the tree.
  - Locate the correct leaf node where the new key should be inserted, maintaining the sorted order.
- **Insert the Key into the Leaf Node:**
  - If the node has **one key**, insert the new key in sorted order.
  - If the node has **two keys**, insert the new key in sorted order, leading to a temporary **three-key node** (overflow).

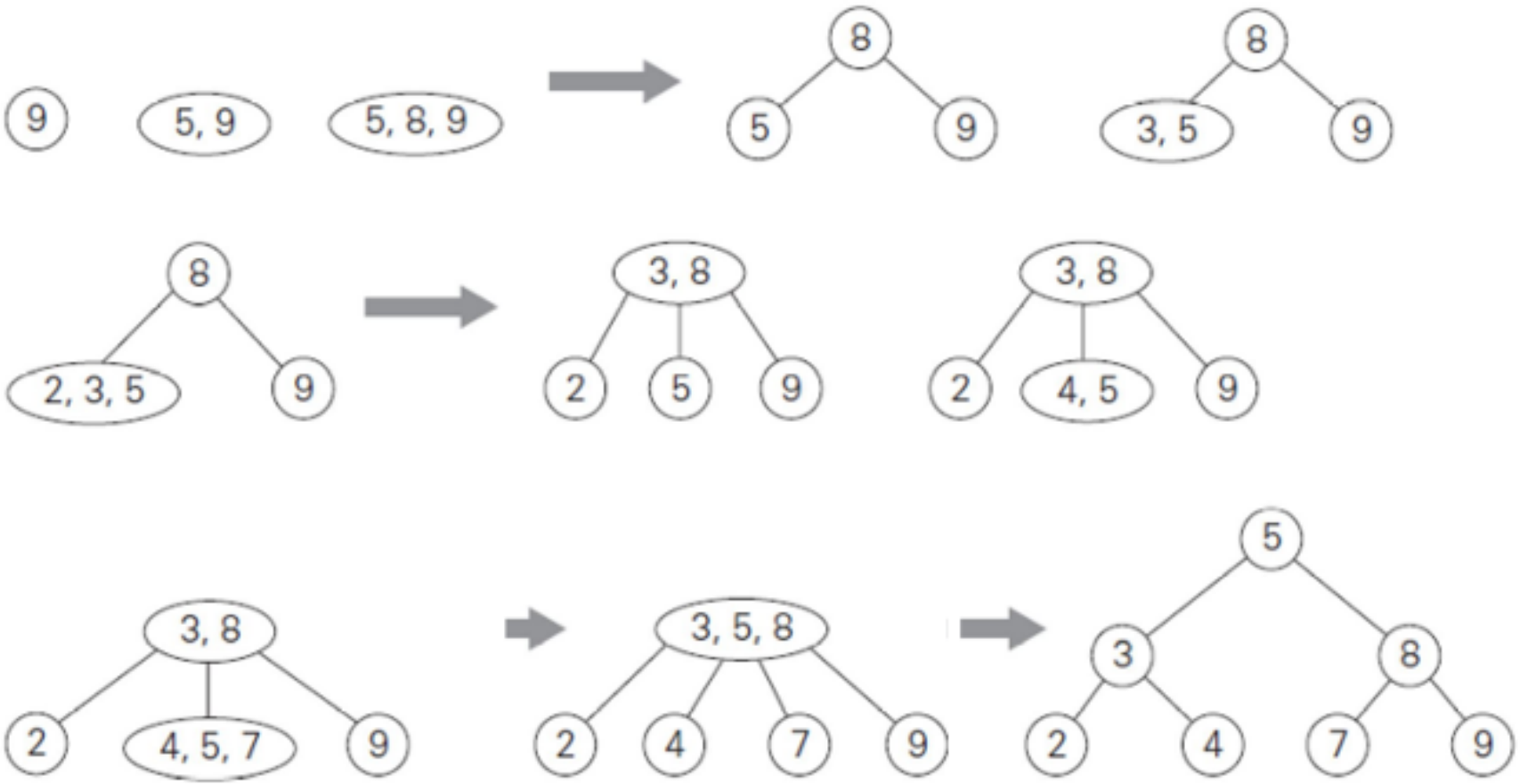
PES UNIVERSITY

- **Handle Overflow (Node Split):**
  - If the leaf node becomes a **three-key node**, split it into two separate nodes:
    - The **middle key** moves up to the parent.
    - The two remaining keys are distributed into two separate nodes.
  - If the parent also becomes a three-key node, repeat the split process up the tree recursively.
- **Grow the Tree Height (If Needed):**
  - If the root splits, create a **new root** with the middle key.
  - The new root will now have two children, increasing the tree height.

Data Structures

2-3 Trees

PES UNIVERSITY



Data Structures

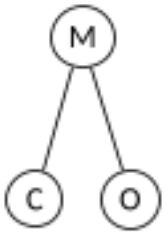
2-3 Trees

PES UNIVERSITY

- Construct a 2-3 tree for the list C, O, M, P, U, T, I, N, G. Use the alphabetical order of the letters and insert them successively, starting with the empty tree.

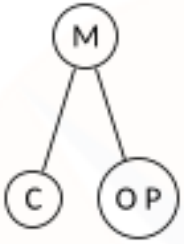
Insert C	Insert O	Insert M
		
The tree is empty, so C becomes the root.	Insert O in sorted order in root	Insert M

Overflow!!



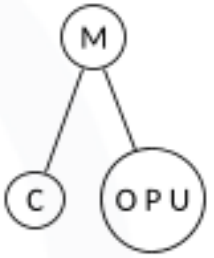
The root already has two keys. Inserting M in sorted order gives [C M O] (temporarily a three-key node/ 4 Node). We split this node

Insert P



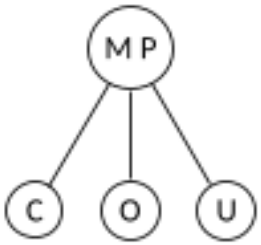
P is greater than M, so we insert it into the right child [O] $\rightarrow$ [O P]

Insert U



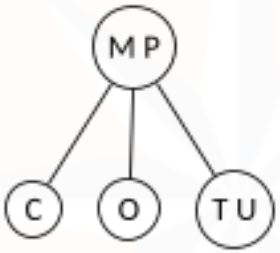
U is greater than P, so we insert it into [O P] $\rightarrow$ [O P U]. This causes an overflow. We split [O P U]

Overflow !!!



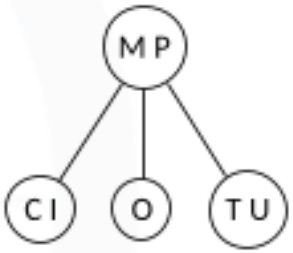
P moves up to the root. O and U remain as separate children.

Insert T



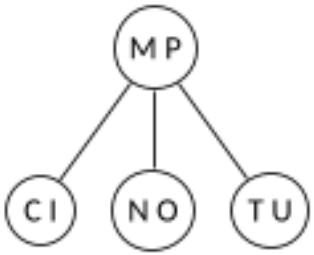
T is greater than P, so it goes into [U] $\rightarrow$ [T U]

Insert I



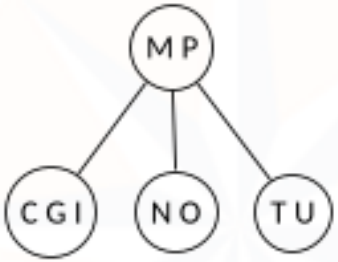
I is less than M, so it goes into [C] $\rightarrow$ [C I]

Insert N



N is between M and P, so it goes into [O] → [N O]

Insert G

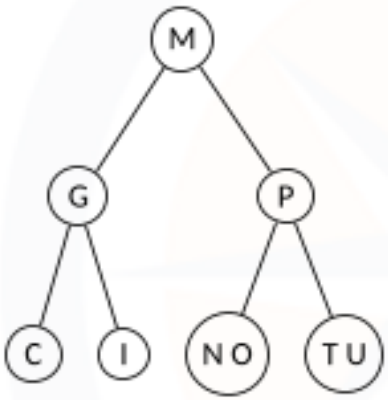


G is less than M, so it goes into [C I] → [C G I]. This causes an overflow, so we split [C G I]

Overflow!!

G moves up to the root. [C] and [I] remain as separate children.

Overflow!!!



M moves up to the root.

[G] and [P] remain as separate children.

Data Structures
Searching in 2-3 Trees

- Let us assume the data element we are searching for is 'd' and the 2-3 tree is 'T'.
- **Empty Tree**
  - If the tree 'T' is empty, then 'd' is not in the tree. The search operation returns **FALSE**.

Searching in 2-3 Trees

- **Value Found in Current Node**
  - If the current node contains a value that is equal to 'd', we return **TRUE**.
  - This indicates that the search operation was successful and the desired value 'd' exists in the tree.

Data Structures
Searching in 2-3 Trees

PES UNIVERSITY

- **Value Not Found at Root Node**
  - If we traverse the tree all the way to a leaf node and it does not contain 'd', we return **FALSE**.
  - This indicates that 'd' is not in the tree.

Searching in 2-3 Trees

- **For a 2-Node**
  - If 'd' is less than the value at the current node, we continue the search operation on the left subtree.
  - If 'd' is greater than the value at the current node, we continue the search operation on the right subtree.

Data Structures
Searching in 2-3 Trees

PES UNIVERSITY

- **Recursive Search**
  - This process continues recursively, navigating down the tree until either the value 'd' is found (and **TRUE** is returned), or it is determined that 'd' is not in the tree (and **FALSE** is returned).

Data Structures

Hands On

PES UNIVERSITY

- Construct a 2-3 Tree by inserting the following sequence of numbers in the given order : 45,30,60,20,50,70,10,40,55,65,80,35,75
  1. Show the final 2-3 Tree after inserting all the elements.
  2. Demonstrate the splitting process when inserting: 60, 70, 80.
  3. What is the height of the final tree?
  4. If 55 is deleted, explain the restructuring process.

Data Structures

Hands On

PES UNIVERSITY

- Construct a 2-3 Tree by inserting the following sequence of numbers in the given order:

  "cat","dog","bat","fox","owl","rat","ant","bee","cow","pig"

  1. Show the final 2-3 Tree after inserting all the words.
  2. Explain the splitting process when inserting: "fox", "owl", "pig"
  3. What is the height of the final tree?
  4. If "bee" is deleted, demonstrate how the tree restructures itself.

Data Structures

Hands On

PES UNIVERSITY

- **Hints to compare Words**
  1. Words are compared the same way as they would appear in a dictionary
  2. Character-by-Character Comparison

     Compare the first letter of each word.

     If the first letter is the same, compare the second letter, and so on.
  3. ASCII Value Comparison OR Alphabet order starting as A-1, B-2, ...

     Words with lower ASCII values come first.

     Shorter Words Precede Longer Words (If Prefix Matches)